

Team Name: The Unemployables

Members: Ayyash A, Tri N, Faris S, Tareq A, Akshayan P.

Date: Aug 4, 2026

1. Features Not Completed

What features did you not complete?

Ayyash: For me personally, I did not complete per-capita donation normalization (showing donations relative to population), shareable/bookmarked URLs so users could save a specific map view, and the organization-level map features (searching for and displaying political organizations on the map).

Akshayan: I was not able to complete the account creation for university-affiliated researchers. Although creating a simple sign-up page was achievable within our sprint time, we didn't end up implementing the full system where the user has role-based access and has to wait for acceptance for the researcher-tier.

Tri: I was unable to complete the trends visualization feature for Electoral Districts. The intended functionality was to display total donations over time (annually or monthly for a selected year) for a specific Electoral District and provide average donation trends across multiple Electoral Districts.

Faris: I was unable to implement the comparison of more than two parties on the riding lookup page.

Tareq: I had initially set my eyes on certain map-linkage features after Demo 2, specifically a "View on map" jump from the Donation Trends and researcher dashboard pages. As a team, we also initially had municipal level filters and per-capita normalization features planned, among others that were reliant on census population data.

Why did you drop them? How did you choose which features to drop?

Ayyash: Per-capita normalization requires importing a separate census population dataset and joining it against the province/riding tables, the data handling was scoped out but time ran out before Demo 3, and it carried over again. Shareable URLs were deprioritized because the core map interaction (year picker, party filter, search, info panel) was higher priority and more visible in demos.

Akshayan: Based on our personas cards, authenticated members require role-based access for individuals based on whether they were university researchers, those in the workforce like elected officials, or just the general public wanting more information. Not only that, but creating the acceptance system for these tiers seemed above the scope and focus of the project, hence we simply went with one Researcher role that had the full set of advanced filtering. By doing so, we were able to focus on improving advanced filtering, exporting records and quick search; features that are far more important than assigning roles. Creating a tier of roles also makes it difficult to split between which individuals get what set of features and which individuals don't, and the acceptance process can be tricky to implement as simply granting individuals access in our Supabase backend wouldn't really ensure there isn't any forgery involved. However, I would say this was more of a priority and time issue rather than an implementation problem.

Tri: I was unable to complete the Electoral District-based trends viewing feature due to time constraints. I implemented a similar trends visualization feature based on provinces instead. The reason I decided to prioritize the provincial-level implementation was because the application relies on pre-aggregated tables to efficiently retrieve donation data. However, Electoral Districts have significantly more entries than provinces, and for certain years, many districts do not receive any donations. This would result in a large number of selectable districts with no available data, reducing the usefulness of the feature and requiring additional time to handle these cases effectively. Therefore, I prioritized implementing the provincial trends view to provide a more complete and reliable user experience within the available timeframe.

Faris: The comparison panel on the Riding Lookup page was build as a fixed two-column layout. Extending it to three or more ridings meant more than just adding a column. Each riding in the comparison is a separate `/api/ridings/:fedNum/summary` request, which means another round trip on page load. It also meant finding an intuitive layout to show all the data without the user being overwhelmed or the page being too crowded. I chose to drop it as the current comparison feature is adequate, and time was better spent on the public-facing pages.

Tareq: My role was centered around authorization and data, and I prioritized the things that posed an actual risk, like session-persistence and authorization logistics. The researcher dashboard was reachable by anyone at first, so I built the protected routes and session persistence first. After that, once the TA's feedback pushed us

toward stability and testing, I made a decision to spend the rest of my efforts on raising test coverage across the whole app instead of adding features. Two reasons: coverage was higher value for the final demo (based on project requirements), and the map/trends linkage would have meant editing code Ayyash and Tri were actively working on, so I'd have been stepping on their features. The map linkage was more of a nice to have, and I don't feel bad about not being able to complete it.

2. Software Engineering Practices

What "best practices" did you adopt? Which ones were effective? Which ones were not effective?

What was effective:

- Since we all had previous experience in maintaining a large project in a group setting, either through CSCB07 or from personal projects, we were able to implement a lot of strategies to help us with completing the project easily and effectively.
- Firstly, we had issue tracking/backlog on GitHub Issues, along with a Google Doc (*CSCC01_Spring_and_Backlog_Planning*) where we broke the product into three epics or main features to implement within our project. Each of these epics had small features, like "Display interactive map," "Create provincial filter," "Implement researcher login". On Github, Faris and Ayyash created these issues and each team member had assigned themselves to their specific issue or added it to ensure they documented the completion of that feature. We added these to the proper demo milestone, allowing us to easily document in our DemoXDocs.md checklist of what we completed during that demo.
- Strict software engineering principles among our code was also strong within our group. Whether that was creating branches for every important feature, using proper naming conventions like "feat/" or "fix/" in branch names and commit messages to easily understand what is happening in the code, or even adding comments that are easy to understand for complex code, we made sure our project followed the guidelines so that the next developer to view our repository wouldn't struggle to identify things. We also required a PR before merging into main to give us a checkpoint to catch integration issues (something that happened quite frequently).
- To our Github, we also added screenshots of our product and of our testing. By testing in vite, we were able to simulate a second-check to ensure our code logic was clear through mock objects, instead of only testing it ourselves.

What wasn't as effective

- Having a backlog on Google Docs and also on Github made things confusing at first.
- We noticed that some of the issues overlapped quite a bit among each other, making our issue board quite messy and generic than what we would have liked it to be.
- We didn't pair program 1:1 a lot since we all seemed to work better individually and we separated our work so that each feature would usually require one team member's attention, and rarely two members. We simply used the Discord group chat to raise any concerns and give updates regarding the project.

Did you as a team adapt or change as the project progressed? If so, how and why?

Yes. As the project progressed, our team evolved from working on independent, low-dependency tasks, such as map rendering, login, the landing page, and database setup, to owning complete features. By Demos 2 and 3, responsibilities became more specialized, with members focusing on areas such as researcher features, map rendering, riding lookup, and trend analysis. We also adapted based on TA feedback by prioritizing improvements such as map colouring, donation lookup performance, and overall polish. We also adapted to technical challenges by upgrading our Supabase database from the free tier to a paid plan because the dataset exceeded the storage limit. After Demo 2, we introduced automated testing with Vite to improve stability and coverage, with each member testing their own features and one member ensuring comprehensive coverage before the final demo. As the team became more familiar with the project and responsibilities were clearly defined, we needed fewer meetings and collaborated mainly to integrate features and meet deadlines. These changes helped us deliver the project successfully while adapting to new challenges throughout development.

In your next project, what would you do differently?

Next time, we will try to look over the backlog harder before creating issues so that we don't duplicate or overlap issues. This created some mess and required a cleanup at the end of our project. Also Since we usually set up meeting times a week in advance, it became quite easy to forget the times or be late to the meetings. Setting up

calendar events to ensure everyone is notified when a meeting is about to start, 10-15 mins beforehand, may reduce some delays.

3. Technical / Design Challenges

What design/solution are you particularly proud of, and why?

Ayyash: The choropleth map's data layer architecture. The map supports selecting any combination of years, filtering by party, and switching between provinces and ridings, all re-colouring the map instantly. Under the hood, three pre-aggregated materialized views (province_year_total, province_party_year_total, riding_year_total) handle the heavy lifting, so the front end gets fast, flat responses regardless of how many years or which party is selected. I was particularly proud of this because I was able to incorporate my knowledge from CSCC43.

Tareq: The protected-routes and session-persistence system for researcher access. When you open or refresh the app, it reads your saved Supabase session before deciding anything, and holds a brief (usually invisible on localhost, due to speed of processing and other factors) "Checking access..." state while it does. That small detail is the part I'm proud of: although it may not be visible on our demo version, without it, a logged-in researcher gets bounced to the login screen for a split second on every refresh, because the session loads asynchronously and looks empty for a moment. I solved it with one shared auth context that both the nav bar and the route guard read from, so the "am I still loading?" question only gets answered in one place. If you're not signed in, the feature sends you to login and then drops you back on the exact page you were trying to reach, instead of sending you to the home or about pages or someplace else.

Akshayan: I was proud of improving my search features over the course of several demos. At the start of my quick search and advanced filtering features for university-affiliated researchers, queries took anywhere between 15-26 seconds. At first, there would be a timeout whenever I would do these complex queries on the website, but not on the Supabase SQL editor. This told me that the roles had a small timeout, so I increased the "authenticated" role timeout for queries from 8 seconds to 30 seconds. This allowed me to finally get the queries without getting the timeout error, but it still took very long. Now, I decided to incorporate more indexes on the table once we got the paid plan for supabase. I added 3 indexes, one based on the first name, one based on the last name and a final index tying in the date, political party and province. This significantly brought down the query time from 26 seconds to nearly 6 seconds. This makes querying a lot faster and more responsive.

Tri: I am proud of how I implemented the donation trends feature. Because the application contains millions of donation records, running aggregate queries on the dataset for every request would have been too slow. Instead, I created pre-aggregated materialized views that store donation summaries by dimensions such as year, province, riding, and political party. This significantly reduced the amount of data that needed to be queried, allowing the trends page to load quickly even under frequent read requests. I am proud of this solution because it provided me with the opportunity to apply system design principles to a real-world problem by optimizing for a read-heavy workload, thereby improving the application's overall performance and scalability.

Faris: I am proud of how the Riding Lookup page was built from the ground up, being slowly improved with each demo. Initially, the riding lookup page just displayed total donation amounts to users. To the average user, this is a meaningless number on its own. Instead of just showing the numbers, the page also tells you the donation average and where the riding ranks out of all 338, to give users an idea of how big a certain riding is.

What was the most difficult technical challenge you faced as a team, and how did you overcome it?

Our biggest technical challenge was achieving fast query performance on a 5.4 million row donations table. Early on, province and riding queries would often time out or return incomplete results because of Supabase's default 1000-row response limit, expensive full-table scans, and the free-tier storage limit. Since this was our first experience working with a dataset of this size, we had to rethink both our database design and deployment strategy. We overcame these issues by upgrading to Supabase Pro and redesigning our database around materialized views that pre-aggregated donation totals by province, riding, year, and party. This reduced expensive million-row scans to a small number of rows per query, eliminated timeouts and truncation issues, and significantly improved the responsiveness of the application. We also created additional materialized views for donor data and other aggregates to further improve performance and the overall user experience.